

# Esquemas Criptográficos: Confidencialidade e Autenticidade

---

## Segurança Informática | UC 3094

---

### Objetivos de Aprendizagem

- Compreender a diferença fundamental entre **criptografia simétrica** e **assimétrica**
  - Identificar os principais algoritmos de cada categoria
  - Compreender como garantir **confidencialidade** através de cifras
  - Entender como garantir **autenticidade** através de assinaturas digitais e MACs
  - Saber escolher o esquema adequado para cada contexto de aplicação
- 

### Criptografia Simétrica: Conceito

- **Chave única** partilhada entre emissor e recetor
  - A mesma chave é usada para **cifrar** e **decifrar**
  - Opera sobre blocos ou fluxos de dados
  - Algoritmos principais:
    - **AES** (Advanced Encryption Standard) — padrão NIST
    - **3DES** (Triple DES) — legado, ainda em uso
    - **ChaCha20** — stream cipher moderna
    - **RC4** — obsoleto, não recomendado
- 

### Criptografia Simétrica: Modos de Operação

| Modo       | Descrição                                          | Uso típico                               |
|------------|----------------------------------------------------|------------------------------------------|
| <b>ECB</b> | Cada bloco cifrado independentemente               | ❌ Não recomendado para dados com padrões |
| <b>CBC</b> | Encadeamento de blocos com IV                      | Armazenamento, comunicação geral         |
| <b>CTR</b> | Contador, transforma block em stream               | Alto desempenho, paralelizável           |
| <b>GCM</b> | Modo autenticado (confidencialidade + integridade) | AES-GCM — padrão moderno                 |

**Aviso:** Nunca reutilizar o mesmo **IV** (Initialization Vector) com a mesma chave!

---

### Criptografia Simétrica: Exemplo (Python)

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os

# Gerar chave e IV
key = os.urandom(32) # AES-256
iv = os.urandom(16) # 128 bits para CBC

# Cifrar
cipher = Cipher(algorithms.AES(key), modes.CBC(iv))
encryptor = cipher.encryptor()
ciphertext = encryptor.update(b"Mensagem secreta") + encryptor.finalize()

# Decifrar
decryptor = cipher.decryptor()
plaintext = decryptor.update(ciphertext) + decryptor.finalize()
```

---

## Criptografia Assimétrica: Conceito

- Par de chaves: **chave pública** (partilhável) e **chave privada** (secreta)
- O que uma chave cifra, a outra decifra
- Algoritmos principais:
  - **RSA** (Rivest-Shamir-Adleman) — baseado na fatoração
  - **ECC** (Elliptic Curve Cryptography) — curvas elípticas, mais eficiente
  - **ElGamal** — baseado no logaritmo discreto
  - **Diffie-Hellman** — acordo de chaves (não cifra diretamente)

---

## RSA: Princípio de Funcionamento

### 1. Geração de chaves:

- Escolher dois primos grandes  $p$  e  $q$
- Calcular  $n = p \times q$  e  $\phi(n) = (p-1) \times (q-1)$
- Escolher  $e$  (expoente público) tal que  $1 < e < \phi(n)$  e  $\gcd(e, \phi(n)) = 1$
- Calcular  $d$  (expoente privado) tal que  $e \times d \equiv 1 \pmod{\phi(n)}$

### 2. Cifra: $C = M^e \bmod n$

### 3. Decifra: $M = C^d \bmod n$

**[FONTE EXTERNA]:** A segurança do RSA baseia-se na dificuldade de fatorar  $n$  em  $p$  e  $q$ .

---

## Comparação: Simétrica vs. Assimétrica

| Característica | Simétrica        | Assimétrica         |
|----------------|------------------|---------------------|
| Chaves         | Única partilhada | Par público/privado |

| Característica   | Simétrica                 | Assimétrica                         |
|------------------|---------------------------|-------------------------------------|
| Desempenho       | ⚡ Muito rápido            | 🐢 100-1000× mais lento              |
| Gestão de chaves | Desafio — partilha segura | Facilitada — chave pública          |
| Tamanho da chave | 128-256 bits              | 2048-4096 bits (RSA)                |
| Exemplo          | AES, ChaCha20             | RSA, ECDSA, ECIES                   |
| Uso típico       | Cifra de dados em volume  | Distribuição de chaves, assinaturas |

## Garantir Confidencialidade

### Com Criptografia Simétrica

- Cifrar o conteúdo com **AES-GCM** ou **ChaCha20-Poly1305**
- Partilhar a chave por um canal seguro
- Utilizar IV aleatório único por operação

### Com Criptografia Assimétrica

- Cifrar com a **chave pública** do destinatário
- Apenas o destinatário (com a chave privada) pode decifrar
- Exemplo: **RSA-OAEP** (Optimal Asymmetric Encryption Padding)

## Garantir Autenticidade

### Message Authentication Code (MAC) — Simétrico

- **HMAC** (Hash-based MAC):  $MAC = H(chave || mensagem)$
- O recetor verifica o MAC com a mesma chave
- Garante **integridade** e **autenticidade** da origem

### Assinatura Digital — Assimétrica

- Gerada com a **chave privada** do emissor
- Verificada com a **chave pública** do emissor
- Garante **integridade, autenticidade e não-repúdio**
- Exemplos: **RSA-PSS, ECDSA**

## Esquemas que Combinam Ambos: Cifra Híbrida

**Motivação:** Eficiência da simétrica + gestão de chaves da assimétrica

1. Gerar chave simétrica temporária (chave de sessão)
2. Cifrar dados com chave simétrica (ex: AES-GCM)
3. Cifrar chave de sessão com chave pública do recetor

```
4. Enviar (ciphertext + chave cifrada + MAC/assinatura)
```

### Exemplos práticos:

- **TLS/SSL** — handshake com assimétrica, dados com simétrica
- **PGP/GPG** — mensagens cifradas e assinadas
- **Hybrid Public Key Encryption (HPKE)** — padrão IETF

[FONTE EXTERNA]

## Exemplo Prático: Envio Seguro com RSA + AES

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os

# 1. Gerar chave de sessão AES
session_key = os.urandom(32)

# 2. Cifrar mensagem com AES-GCM (confidencialidade + autenticidade)
iv = os.urandom(12)
cipher = Cipher(algorithms.AES(session_key), modes.GCM(iv))
encryptor = cipher.encryptor()
ciphertext = encryptor.update(b"Mensagem confidencial") + encryptor.finalize()

# 3. Cifrar chave de sessão com RSA pública do recetor
cipher_rsa = public_key.encrypt(
    session_key,
    padding.OAEP(mgf=padding.MGF1(hashes.SHA256()), algorithm=hashes.SHA256())
)

# Enviar: cipher_rsa + iv + ciphertext + tag
```

## Resumo: Escolha do Esquema

| Contexto                   | Esquema Recomendado                     |
|----------------------------|-----------------------------------------|
| Dados em repouso (storage) | <b>AES-GCM</b> ou <b>AES-XTS</b>        |
| Comunicação ponto-a-ponto  | <b>TLS</b> (híbrido)                    |
| Assinatura de documentos   | <b>RSA-PSS</b> ou <b>ECDSA</b>          |
| Troca segura de chaves     | <b>Diffie-Hellman</b> ou <b>RSA-KEM</b> |
| Autenticação de mensagens  | <b>HMAC-SHA256</b> ou <b>AES-GCM</b>    |

| Contexto              | Esquema Recomendado                           |
|-----------------------|-----------------------------------------------|
| Cifra com não-repúdio | <b>Assinatura digital</b> + cifra assimétrica |

---

## Síntese: Pontos-Chave

- **Criptografia simétrica** — rápida, chave única, ideal para dados em volume
  - **Criptografia assimétrica** — par de chaves, mais lenta, ideal para distribuição de chaves e assinaturas
  - **Confidencialidade** — obtida com cifra (simétrica ou assimétrica)
  - **Autenticidade** — obtida com MAC (simétrico) ou assinatura digital (assimétrica)
  - **Esquemas híbridos** — combinam o melhor de ambos os mundos
  - A escolha do esquema depende do **contexto de aplicação** e dos requisitos de **segurança**
- 

## Bibliografia

- Gollmann, D. (2011). *Computer Security*, 3rd Edition. Wiley. ISBN 9780470741153
- Du, W. (2017). *Computer Security: A Hands-on Approach*. CreateSpace. ISBN 9781548367947
- Ferguson, N., Schneier, B., & Kohno, T. (2010). *Cryptography Engineering*. Wiley. [FONTE EXTERNA]
- Katz, J., & Lindell, Y. (2014). *Introduction to Modern Cryptography*. CRC Press. [FONTE EXTERNA]